

Ch 10 Deep Learning ← Huge!

Covers 3 model types: Convolutional recurrent

Neural Networks (NN)

↳ < 1980s

↳ ML reg/class techn
on par with other
methods

↳ Best for large data
+ highly nonlinear

Details now!

CNN

↳ Image

↳ 2010+

↳ Better than
other ML

RNN

↳ Sequence data

↳ Text data

↳ 2010+

Deep Learning (in text)

Lots of notation & terminology.

2020

Neural networks (Regression case)

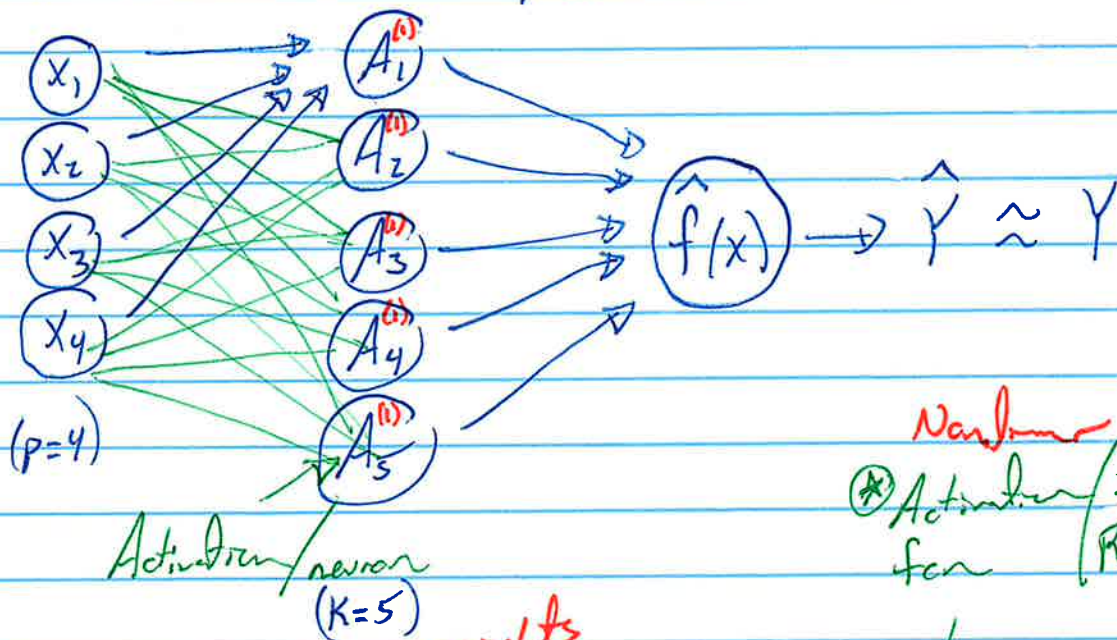
pg 2

$$Y = f(X) + \epsilon, \quad X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{bmatrix} \quad \text{Goal: Approx } f(X)$$

Input layer

Hidden layers (L=1)

NN graph
viz



Nonlinear
* Activation function (sigmoid, ReLU)

$$A_1 = h_1(X) = g(\underbrace{w_{10} + w_{11}X_1 + \dots + w_{14}X_4}_{\text{weights}}) = g(\underbrace{w_{10}}_{\text{Bias}} + \sum_{j=1}^4 w_{1j}X_j)$$

for K $\rightarrow$ $A_K = h_K(X) = g(\underbrace{w_{K0}}_{\text{Bias}} + \sum_{j=1}^p \underbrace{w_{Kj}X_j}_{\text{LC of } X})$ (fit weights to train data)

$$\hat{f}(X) = \beta_0 + \beta_1 A_1 + \beta_2 A_2 + \dots + \beta_5 A_5 = \beta_0 + \sum_{K=1}^5 \beta_K A_K$$

(fit β_K to train data) $\rightarrow$ min RSS as before.

classification $\rightarrow$ 2 classes use $\hat{f}$ as sigmoid
3+ classes use softmax (as in multinomial logistic reg)

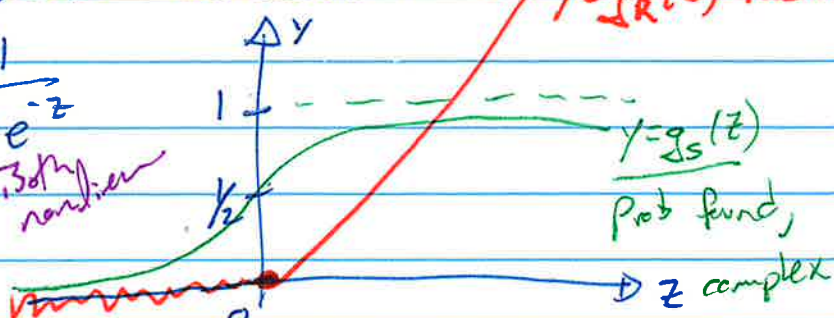
Activation functions $g(z) = ?$

Many choices, 2 most common.

Simplest nonlinear fun PS 3

tanh(x)

Sigmoid: $g(z) = \frac{1}{1+e^{-z}}$
 * 3rd most nonlinear



Lowered 2000+

Rectified linear unit.

ReLU $g_R(z) = \begin{cases} z, & z \geq 0 \\ 0, & z < 0 \end{cases} = \text{ReLU}(z)$

Idea: 2+ layers in NN

For composition $g(g(\cdot))$



$$A_k^{(2)} = g\left(w_{k0}^{(2)} + \sum_i w_{ki}^{(2)} A_{ki}^{(1)}\right)$$

grows nonlinear complexity. NN adv

10.1

Fitting a NN (single layer case)
 $p=4, K=5$
 regression

10.7

$$\hat{f}(x) = \beta_0 + \sum_{k=1}^K \beta_k A_k = \beta_0 + \sum_{k=1}^K \beta_k g(z_k)$$

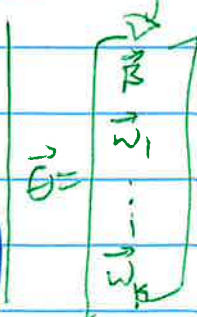
$$= \beta_0 + \sum_{k=1}^K \beta_k g\left(w_{k0} + \sum_{j=1}^p w_{kj} x_j\right)$$

$\beta_s = 6$
 $w's = 5.5$
 31 total 31 comp.

To fit $\hat{f}$ to train data $(x_i, y_i), i=1, \dots, n$
 choose β_i 's & w_{kj} 's which

Solve optm via gradient descent.

$\min_{\vec{\beta}, \vec{w}_k} (RSS) = \min_{\vec{\beta}, \vec{w}_k} \left(\frac{1}{2} \sum_{i=1}^n (y_i - \hat{f}(x_i))^2 \right)$
 as before



Cost fun $\rightarrow R(\vec{\theta}), \vec{\theta}$ stack of $\vec{\beta}, \vec{w}_k$

Calc 1 version of G.W. $R(\theta)$, ∂R

Local min

Global min

$y = R(\theta)$

Find min of $R(\theta)$

Tangent slope $R'(\theta_0) = 0$

θ_1 , θ_2 , θ_3

General formula: $\Theta^{k+1} = \Theta^k - \underbrace{\rho R'(\Theta^k)}_{\text{Gradient descent}}$

Genral gradient descent

θ^0 initial guess

$$\theta^0 = \begin{bmatrix} \vec{w}_1 \\ \vdots \\ \vec{w}_k \end{bmatrix} \in \mathbb{R}^n$$

Repeat until converges

Update: $\theta^{k+1} = \theta^k - \rho \nabla R$

Update

$$\nabla R = \begin{bmatrix} \frac{\partial R}{\partial \theta_1} \\ \vdots \\ \frac{\partial R}{\partial \theta_n} \end{bmatrix}$$

$$\frac{\partial R}{\partial \theta_i} = 1^{\text{st}} \text{ partial deriv}$$

other vars const

Gradient vector (direction of steepest increase)

- GD issues:
- ① Often finds local min rather than global min
 - ② $\vec{\theta}$ & $\vec{w}_k$ updates involve many calculations in GD, usually slow, instead process batches (subset) of train data per iteration.

